

GrowthOS Backend Architecture

Meta Graph API Integration Blueprint

Role: Senior Backend Architect & Meta Integration Engineer

Target Stack: Node.js, Express, MongoDB, Mongoose

Document Scope

- Systematic Technical Flow
- Auth & Token Architecture
- Database Modeling
- Security & Error Handling
- Incremental Development Plan

PART 1 — Understand the Existing GrowthOS Backend

Before introducing Meta, we must establish the boundaries of the existing system based on GrowthOS documentation.

- **Stack:** Node.js, Express.js, MongoDB via Mongoose.
- **Internal Auth:** JWT-based authentication. The user logs in via GrowthOS frontend, receives a JWT, and sends it via the `Authorization: Bearer <JWT>` header.
- **Middleware:** A `protect` middleware validates the JWT and attaches the user document to `req.user`.
- **Error Handling:** Standardized via an `asyncHandler` wrapper around controllers (No explicit `try/catch` blocks inside controllers).
- **Data Model:** Existing `User` and `Tenant` models.

Architectural Boundary Confirmed

Meta integration will act as an **external data provider**. It does *not* replace GrowthOS authentication. A user must already be authenticated in GrowthOS to connect their Meta account.

PART 2 — The Core Concept: Two Authentication Systems

It is critical to distinguish between our internal authentication and Meta's authorization system.

GrowthOS JWT

- **Purpose:** Identifies and authenticates the user *inside* GrowthOS.
- **Source:** GrowthOS login/register endpoints.
- **Usage:** Protects GrowthOS backend routes.

Meta Access Token

- **Purpose:** Represents a user's authorization to access permitted Meta resources via the Graph API.
- **Source:** Meta OAuth 2.0 flow.
- **Usage:** Appended to requests made from the GrowthOS backend to `graph.facebook.com`.

The Relationship:

GrowthOS User → GrowthOS JWT → Authenticated GrowthOS Backend Request → Meta OAuth Connection → Meta Access Token → Meta Graph API Resources.

PART 3 — Complete End-to-End User Flow

Here is the detailed flow for a user with no existing Meta connection, broken down by architectural steps.

Step	Initiator / Actor	Action Details (Tokens, Data, Storage, Security)
1. GrowthOS Login	User / Frontend	User logs into GrowthOS. Backend issues a GrowthOS JWT. Frontend stores JWT.
2. Init Meta Connect	User / Frontend	User clicks "Connect Meta". Frontend calls GrowthOS Backend (<code>GET /api/meta/connect</code>). Request includes JWT (Bearer).
3. Generate OAuth URL	Backend	Backend validates JWT (via protect). Generates a unique <code>state</code> parameter (stored in cache/session tied to user). Constructs Meta OAuth URL with Client ID, Redirect URI, Scopes. Responds to Frontend with URL.
4. Meta Redirect	Frontend / Browser	Frontend redirects browser to Meta OAuth URL. No GrowthOS tokens involved here. User interacts directly with Meta.
5. Authorization	User / Meta	User logs into Meta, reviews requested scopes, and approves. <i>Failure point: User denies access.</i>
6. Callback to GrowthOS	Meta / Browser	Meta redirects browser to GrowthOS callback URL (e.g., frontend route or direct backend route depending on architecture), passing <code>code</code> and <code>state</code> .
7. State Validation	Backend	Backend receives <code>code</code> and <code>state</code> . Verifies <code>state</code> matches the one generated in Step 3 to prevent CSRF attacks.
8. Token Exchange	Backend	Backend makes server-to-server POST to Meta with <code>code</code> , App ID, and App Secret. Returns Short-Lived User Access Token. <i>Failure: Invalid code/secret.</i>
9. Token Upgrade	Backend	Backend makes server-to-server request to exchange short-lived token for Long-Lived User Access Token (valid for 60 days).
10. Store Connection	Backend	Backend saves Long-Lived token, Meta User ID, and granted scopes in <code>MetaIntegration</code> document linked to <code>req.user._id</code> .
11. Fetch Assets	Backend	Backend uses the token to query Meta for available assets (Pages, Ad Accounts). Stores asset IDs.
12. Return Status	Backend	Backend responds to frontend: <code>{ status: 'connected', assets: [...]</code> }. Frontend updates Dashboard.

PART 4 — Meta OAuth Flow in Detail

The OAuth flow must be handled securely. The App Secret must never be exposed to the frontend.

- **Meta Developer App:** The container in the Meta App Dashboard holding configurations.
- **App ID & App Secret:** Credentials linking your code to the Meta App. Secret is strictly Backend-only.
- **Redirect URI:** An exact, strictly-matched URL registered in the Meta App Dashboard.
- **Scopes:** Comma-separated list of permissions (e.g., `pages_read_engagement, ads_read`).
- **State Parameter:** A backend-generated random string to prevent CSRF.
- **Authorization Code:** A short-lived (usually 10 mins) code returned by Meta to the browser.

```
Frontend                                GrowthOS Backend                                Meta OAuth Server
|                                     |                                     |
|-- GET /api/meta/connect --->|                                     |
| (Bearer JWT)                  |                                     |
|                               |-- Generate State                    |
|<-- Return Auth URL -----|                                     |
|                               |                                     |
|-- Redirect browser to Meta Auth URL ----->|                                     |
|                               |                                     | (User approves)
|<-- Redirect to Callback URI with ?code=XYZ & state=ABC -----|                                     |
|                               |                                     |
|-- GET /api/meta/callback -->|                                     |
| ?code=XYZ&state=ABC          |                                     |
|                               |-- Validate State                    |
|                               |-- POST /oauth/access_token --->| (Sends Code + App
Secret)                           |                                     |
|                               |<-- Returns Access Token -----|                                     |
|                               |                                     |
|                               |-- Store Token in DB                |                                     |
|<-- Return Success -----|                                     |
```

Architectural Recommendation

The authorization code exchange **must** happen on the backend. If the frontend attempts this, it requires the App Secret, which completely compromises the security of your Meta App.

PART 5 — Token Architecture

A single Meta access token does NOT automatically give unlimited access to every Meta resource. Access is highly specific.

- **GrowthOS JWT:** Obtained on login. Stored in Frontend. Authorizes GrowthOS API calls. Expires based on your `JWT_EXPIRE` env var.
- **Meta User Access Token (UAT):** Obtained via OAuth. Stored in DB securely. Authorizes actions on behalf of the specific Meta User. Upgradable to a Long-Lived Token (60 days).
- **Meta Page Access Token (PAT):** Obtained by querying `/me/accounts` using the UAT. Stored in DB. Required to read Page insights, post to pages, or read comments. Never expires natively if generated from a Long-Lived UAT, but can be invalidated.
- **System User Token:** (If using Business Manager APIs heavily - usually not needed for basic SaaS integrations unless verified as a Meta Business Partner).

Note: A token's power is dictated by its **scopes**, the **user's role** on the asset, and the **App's approval status** (App Review).

PART 6 — Object IDs and Graph API Requests

Tokens answer "Who is authorized?" Object IDs answer "Which resource am I accessing?"

The Graph API is built on nodes (Objects) and edges (Connections). To get insights for a campaign, you need the Campaign ID, not just the Ad Account ID.

```
Meta Access Token (UAT or PAT)
├── Page ID (Node) —————> /v19.0/{page-id}/insights
├── Ad Account ID —————> /v19.0/act_{ad-account-id}/campaigns
└── Instagram Account ID -> /v19.0/{ig-account-id}/insights
```

PART 7 — "What Insights Do We Actually Need?"

Do NOT assume "user insights" means personal profile data. In B2B SaaS, this almost always means Business/Marketing insights. **Clarification with Product Team is Mandatory.**

Category	Object Required	Likely Token Needed	Required Scopes (Subject to check)	Example Metrics	Action Needed
----------	-----------------	---------------------	------------------------------------	-----------------	---------------

FB Page Data	Page ID	Page Access Token	<code>pages_read_engagement</code>	Page likes, impressions, reach	Verify if SaaS targets Organic Social.
Ad Performance	Ad Account ID	User Access Token	<code>ads_read</code>	Spend, ROAS, CPC, CTR	Verify if SaaS targets Paid Media.
IG Business	IG User ID	User Access Token	<code>instagram_basic,</code> <code>instagram_manage_insights</code>	Profile views, follower growth	Verify if IG is included in scope.

Assumption: For GrowthOS, it's highly likely we need Ads and Page insights. I will plan for extensible asset architecture.

PART 8 — Graph API Request Architecture

Do NOT dump Meta API communication directly into `metaController.js`. Use a Service Layer pattern.

- **Route** (`metaRoutes.js`): Defines endpoints (e.g. `/api/meta/insights`). Applies `protect` middleware.
- **Controller** (`metaController.js`): Wrapped in `asyncHandler`. Extracts `req.user._id`. Calls Service layer. Sends standardized JSON responses.
- **Service** (`metaService.js`): Holds Meta-specific logic. Uses Axios/Fetch to call `graph.facebook.com`. Manages token injection, versioning (v19.0), and Meta-specific error parsing.
- **Model** (`MetaIntegration.js`): Mongoose schema abstracting DB operations.

PART 9 — Database Design

Recommendation: Option B (Separate `MetaIntegration` model). Storing this in the `User` model creates bloat, mixes concerns, and makes handling multiple integrations (Meta, Google, TikTok) chaotic.

```
// Conceptual Schema: MetaIntegration
{
  userId: { type: ObjectId, ref: 'User', required: true, unique: true },
  metaUserId: { type: String, required: true },
  accessToken: { type: String, required: true }, // Long-lived UAT
  tokenExpiresAt: { type: Date },
  grantedScopes: [{ type: String }],
  status: { type: String, enum: ['connected', 'disconnected', 'expired'], default:
'connected' },

  // Discovered Assets
  adAccounts: [{
    accountId: String,
    name: String,
    isActive: Boolean
  }],
  pages: [{
    pageId: String,
    name: String,
    pageAccessToken: String // Some APIs require the PAT
  }],

  createdAt: Date,
  updatedAt: Date
}
```

Rules: One GrowthOS User maps to One Meta Connection. One Meta Connection can have Multiple Pages/ Ad Accounts.

PART 10 — API Routes to Build

Below is the recommended RESTful route structure.

Endpoint	Method	Auth	Purpose / Responsibility
<code>/api/meta/connect</code>	GET	JWT	Generates OAuth URL + state. Controller initiates flow.
<code>/api/meta/callback</code>	GET	JWT	Receives code/state. Service exchanges code for token, stores DB.
<code>/api/meta/status</code>	GET	JWT	Checks if user has active Meta connection in DB.
<code>/api/meta/assets</code>	GET	JWT	Service calls Meta API for Pages/AdAccounts and syncs to DB.
<code>/api/meta/insights</code>	GET	JWT	Takes <code>assetId</code> and <code>metric</code> in query. Service queries Graph API.
<code>/api/meta/disconnect</code>	DELETE	JWT	Revokes token via Graph API, marks DB status as disconnected.

PART 11 — Security Architecture (CRITICAL)

- **App Secret Protection:** `META_APP_SECRET` goes in `.env`. NEVER committed to Git. Never sent to frontend.
- **State Validation:** Mitigates CSRF. The state generated in `/connect` must match the state received in `/callback`.
- **Tenant Isolation:** Controllers must query `MetaIntegration.findOne({ userId: req.user._id })`. Never allow a user to pass a `userId` in the body.
- **Token Security:** Do not log Meta access tokens. In production, consider encrypting tokens at rest in MongoDB using Mongoose field-level encryption. Do not return raw tokens to the frontend dashboard.
- **Least Privilege:** Only request scopes explicitly required by the Product requirements.

PART 12 — Error Handling Mapping

Failure Scenario	Detected Where?	HTTP Status	Safe Frontend Response	Internal Action
User denies OAuth access	Callback (Meta sends <code>error</code>)	400	"Authorization cancelled by user."	Log warning. Do not create DB record.
Invalid/Expired OAuth Code	Service (Token Exchange)	401	"Connection failed. Please try again."	Log Meta API error.
Token Expired / Revoked	Service (Graph API call)	401	"Meta connection expired. Reconnect."	Update DB status to 'expired'.
Missing Permissions	Service (Graph API call)	403	"Insufficient Meta permissions for asset."	Log missing scope. Prompt user re-auth.
Invalid Asset ID	Service (Graph API call)	404	"Asset not found or access denied."	Log ID failure.

PART 13 — Development Plan

Incremental stages to prevent overwhelming the architecture.

- **STAGE 0: Repo Verification** - Review `protect` middleware and `asyncHandler` implementation.
- **STAGE 1: Requirement Clarification** - Meet with Product to define EXACTLY which metrics (Pages vs Ads vs IG) are needed.
- **STAGE 2: Meta API Research** - Map the required metrics to Graph API v19.0 endpoints and scopes.

- **STAGE 3: Meta App Setup** - Create App, configure App ID, Secret, and OAuth Redirect URIs in Meta Dashboard.
- **STAGE 4: Environment Config** - Add variables to `.env`.
- **STAGE 5: DB Design** - Create `MetaIntegration` Mongoose model.
- **STAGE 6 & 7: OAuth Endpoints** - Implement `/connect` and `/callback`.
- **STAGE 8: Token Exchange** - Implement backend HTTP call to exchange code for token.
- **STAGE 9: Asset Discovery** - Implement logic to fetch Ad Accounts / Pages and save to DB.
- **STAGE 10: Graph Service** - Build the `metaService.js` wrapper.
- **STAGE 11: Insights Endpoint** - Implement specific data-fetching endpoints.
- **STAGE 12: Postman Testing** - Test valid, invalid, and expired flows.
- **STAGE 13: Security Hardening** - Review for tenant isolation and secret leakage.
- **STAGE 14: Documentation & Handoff** - Document endpoints for Frontend team.

PART 14 — Development Progress Tracker

Stage	Task	Status	Evidence/Notes
0	Repo verified	[]	
1	Requirements confirmed	[]	
2	Meta docs researched	[]	
3	Meta App created	[]	
4	OAuth configured	[]	
5	Meta model created	[]	
6	Connect endpoint	[]	
7	Callback endpoint	[]	
8	Token exchange	[]	
9	Assets fetched	[]	
10	Graph service built	[]	
11	Insights fetched	[]	
12	Postman tests complete	[]	

13	Security review	[]	
14	Documentation	[]	

PART 15 — Git / Development Workflow

Work on a feature branch (e.g., `feature/meta-integration`). Commit logically:

- `chore(env): add meta environment variables`
- `feat(meta): create MetaIntegration mongoose model`
- `feat(meta): implement oauth connect and callback routes`
- `feat(meta): implement token exchange in meta service`
- `feat(meta): add asset discovery and insights routes`
- `fix(meta): handle expired graph api token errors`
- `docs(meta): add postman collection for meta endpoints`

Ensure local testing passes before opening a PR. No secrets in PR diffs.

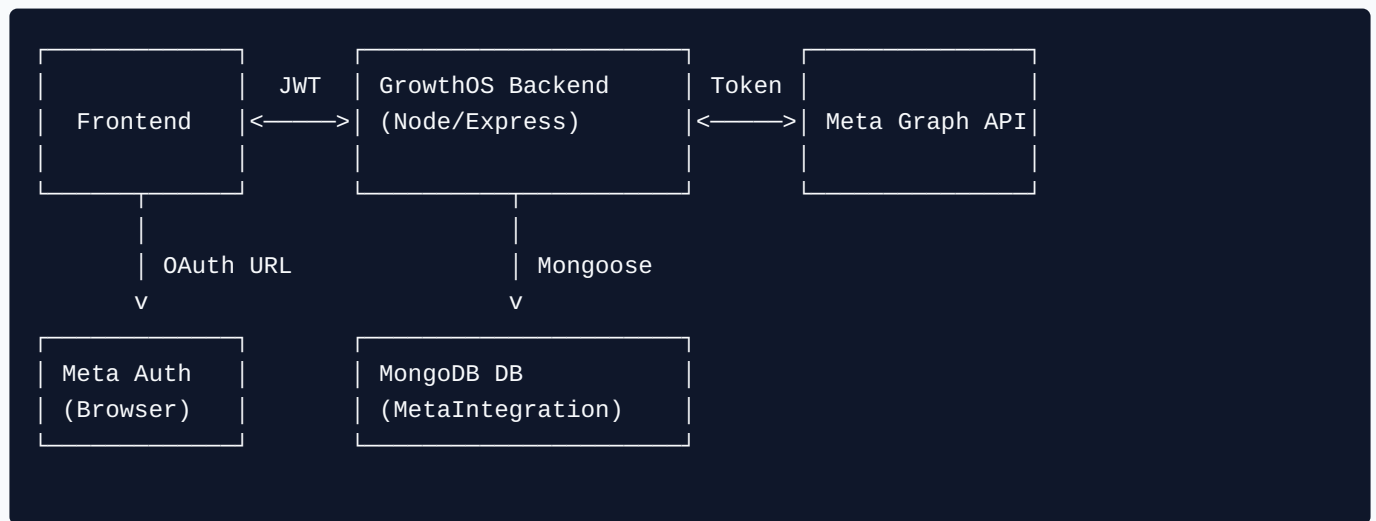
PART 16 — Postman Testing Flow

Testing OAuth requires a mix of Postman and Browser.

1. **Postman:** POST `/api/auth/login` -> Copy GrowthOS JWT.
2. **Postman:** GET `/api/meta/connect` (Header: Bearer JWT) -> Copy returned URL.
3. **Browser:** Paste URL, login to Meta, authorize app.
4. **Browser/Network Tab:** Catch the redirect to `/api/meta/callback?code=XYZ...`
5. **Postman:** Manually trigger the callback endpoint with the code (if redirect is frontend-led), OR verify DB record is created if callback is backend-led.
6. **Postman:** GET `/api/meta/status` -> Verify "connected".
7. **Postman:** GET `/api/meta/assets` -> See list of IDs.
8. **Postman:** GET `/api/meta/insights?assetId=123` -> Verify Meta data returns formatted for GrowthOS.

PART 17 — Architecture Diagrams

A. End-to-End System Architecture



B. Backend Request Lifecycle

```
Client Request
  |
  v
[ Route Layer ] -> protect (Validates JWT, sets req.user)
  |
  v
[ Controller ] -> asyncHandler wrapper (No try/catch)
                  Extracts inputs, calls Service
  |
  v
[ Service Layer ] -> Retrieves DB Token, Calls Axios to Meta API, Parses response
  |
  v
[ JSON Response ] <- Controller returns standardized 200 OK
```

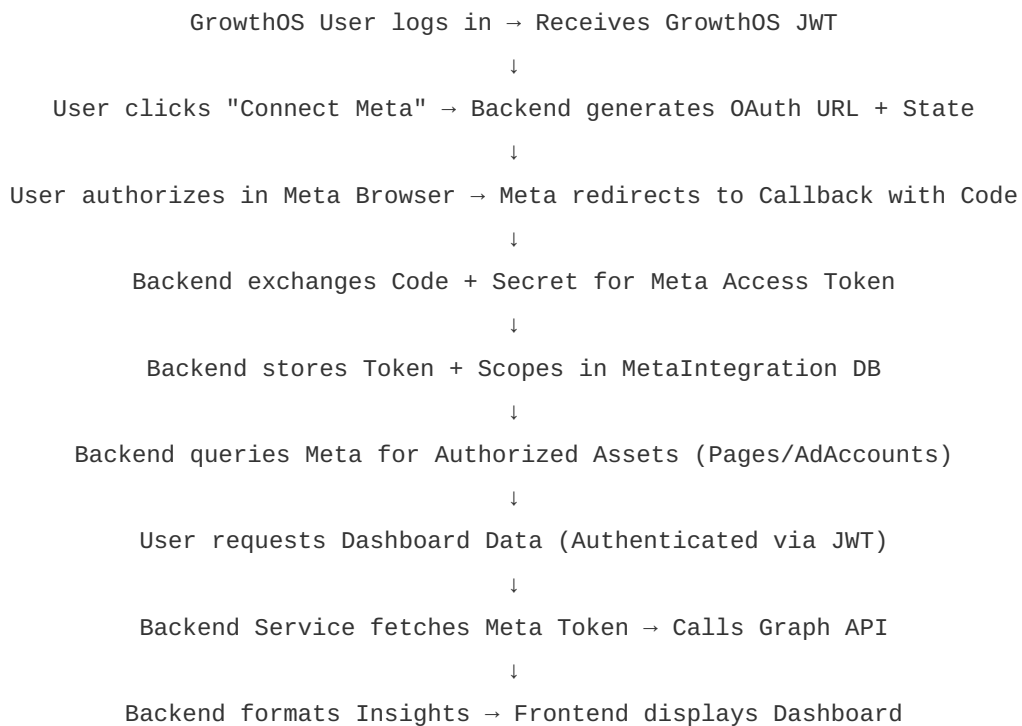
PART 18 — Common Mistakes to Avoid (WHAT NOT TO DO)

- ❌ Treating Meta access token as GrowthOS JWT.
- ❌ Putting `META_APP_SECRET` in the frontend or committing it.
- ❌ Assuming OAuth success means all assets are accessible (User might have no Ad Accounts).
- ❌ Assuming one permission grants everything (Scopes are granular).
- ❌ Putting all Axios/Meta API logic directly into Express controllers.

- ❌ Storing everything directly in the `User` model.
- ❌ Returning raw Meta Graph API responses directly to the frontend (normalize the data first).
- ❌ Implementing fetching logic before confirming the EXACT business requirement with Product.

PART 19 — Final Implementation Blueprint

MASTER EXECUTION FLOW



MY DEVELOPMENT FLOW

Understand Repo → Clarify Requirements → Research Meta Docs → Configure Meta App → Design DB Schema → Implement OAuth Connect → Implement Token Exchange → Discover Assets → Build Graph API Service → Implement Insights → Test via Postman → Secure & Handle Errors → Document → PR / Team Handoff.